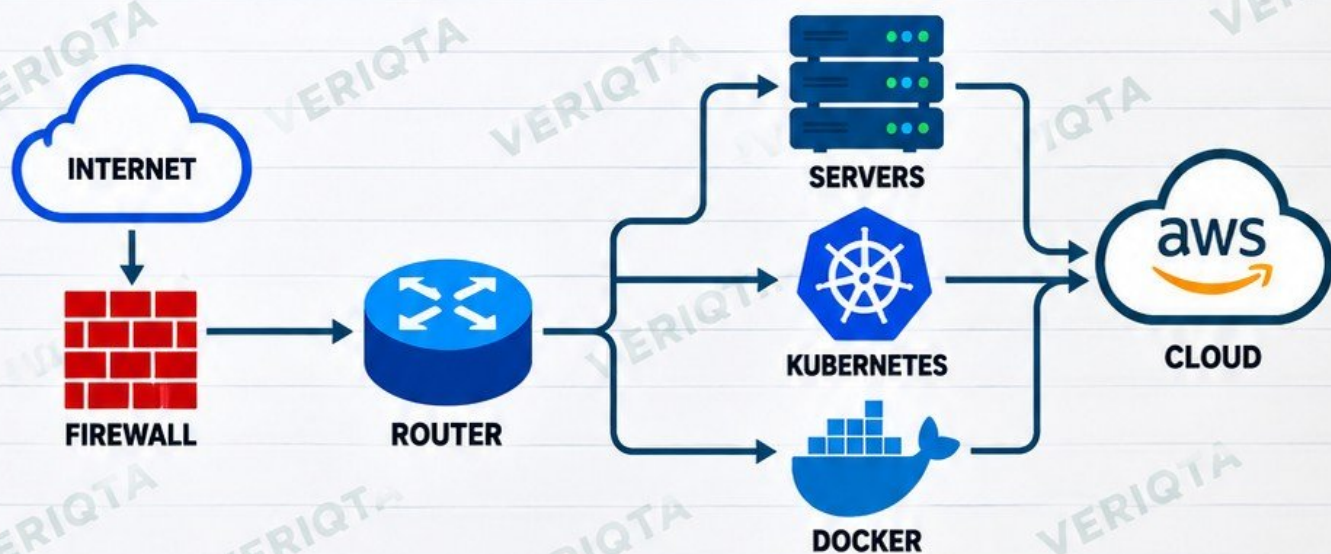


NETWORKING PROJECTS 20 FOR DEVOPS ENGINEERS

BUILD | CONFIGURE | TROUBLESHOOT | SECURE | AUTOMATE

HANDS-ON NETWORKING LABS FOR REAL-WORLD INFRASTRUCTURE



PRACTICE TODAY
DEPLOY TOMORROW

BY VERIQTA

LEARN | BUILD | GROW

Build a Linux Network Troubleshooting Lab

Create isolated Linux hosts with network namespaces and veth pairs. Inspect interfaces, routes, ARP/neighbor tables, sockets, and traffic using **ip**, **ss**, **ping**, **traceroute**, and **tcpdump**.

PROJECT GOAL

- Set up isolated Linux hosts using network namespaces.
- Connect them with veth pairs.
- Explore real network behavior.
- Practice troubleshooting with production-grade tools.

WHAT YOU WILL BUILD

- ✓ 3 isolated Linux hosts (namespaces)
- ✓ Virtual network using veth pairs
- ✓ IP addressing and routing
- ✓ Test connectivity and failures
- ✓ Capture and analyze traffic
- ✓ Investigate ARP, routes, sockets

TOOLS USED

- **ip** (iproute2)
- **ss** (socket statistics)
- **ping**
- **traceroute** / **tracpath**
- **tcpdump**
- **nsenter**
- Optional: **wireshark**

LAB TOPOLOGY



1. CREATE NETWORK NAMESPACES

```
sudo ip netns add ns1
sudo ip netns add ns2
sudo ip netns add ns-router
```

2. CREATE VETH PAIRS

```
sudo ip link add veth1 type veth \
peer name veth1-r
sudo ip link add veth2 type veth \
peer name veth2-r
```

3. MOVE INTERFACES TO NAMESPACES

```
sudo ip link set veth1 netns ns1
sudo ip link set veth1-r netns ns-router
sudo ip link set veth2 netns ns2
sudo ip link set veth2-r netns ns-router
```

4. CONFIGURE IP ADDRESSES

```
sudo ip netns exec ns1 ip addr add
10.10.1.1/24 dev veth1
sudo ip netns exec ns1 ip link set
veth1 up
sudo ip netns exec ns1 ip route add
default via 10.10.1.254
```

5. ENABLE IP FORWARDING (ROUTER)

```
sudo ip netns exec ns-router \
sysctl -w net.ipv4.ip_forward=1
# Add router IPs
sudo ip netns exec ns-router \
ip addr add 10.10.1.254/24 dev veth1-r
sudo ip netns exec ns-router \
ip addr add 10.10.2.254/24 dev veth2-r
sudo ip netns exec ns-router ip link set veth1-r up
sudo ip netns exec ns-router ip link set veth2-r up
```

6. TEST CONNECTIVITY

```
sudo ip netns exec ns1 ping 10.10.2.1
sudo ip netns exec ns1 traceroute
10.10.2.1
sudo ip netns exec ns2 ping 10.10.1.1
```

7. INSPECT NETWORK STATE

```
sudo ip netns exec ns1 ip a
sudo ip netns exec ns1 ip r
sudo ip netns exec ns1 ip neigh
sudo ip netns exec ns1 ss -tunap
```

8. CAPTURE AND ANALYZE TRAFFIC

```
sudo ip netns exec ns-router tcpdump -i any
sudo ip netns exec ns1 tcpdump -i veth1
sudo ip netns exec ns2 tcpdump -i veth2
# In another terminal generate traffic
sudo ip netns exec ns1 ping 10.10.2.1
```

9. EXPERIMENT AND BREAK THINGS

- Remove a route and observe failure
- Change MTU and see packet issues
- Block traffic with iptables
- Observe ARP cache behavior
- Capture connection attempts with tcpdump
- Analyze TCP handshakes and resets



KEY TAKEAWAY

Network issues are easier to solve when you can reproduce them.



REAL WORLD SKILL

This lab builds the foundation for troubleshooting containers, Kubernetes and cloud networks.



NEXT STEP

Extend this lab with DNS, firewall rules and NAT in the next projects.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta


VERIQTA
LEARN | BUILD | GROW

Trace a Packet End to End

Follow an application request through DNS resolution, routing, ARP/neighbor discovery, TCP handshake, application traffic, and response.

Capture the flow with **tcpdump** and **Wireshark**.

PACKET FLOW OVERVIEW



PROJECT GOAL

- Trace a real application request end to end.
- Understand each network layer.
- Capture and analyze packets with **tcpdump**.
- Inspect the traffic in **Wireshark**.
- Learn how to identify issues at each stage.

TOOLS USED

- dig** (DNS queries)
- ip**, **arp**, **route** (network info)
- ping**, **tracert**, **mtr** (path test)
- tcpdump** (packet capture)
- Wireshark** (packet analysis)
- curl** (generate traffic)
- ss**, **netstat** (socket info)

WHAT YOU WILL LEARN

- How DNS resolution works
- How routing selects the next hop
- How ARP finds the destination MAC
- How the TCP handshake is established
- What application traffic looks like
- How the response travels back
- How to capture and analyze packets
- How to troubleshoot network issues

STEP-BY-STEP LAB

1 DNS RESOLUTION (Find the IP)

```
dig example.com
dig example.com +trace
```

2 CHECK ROUTING (Find the Path)

```
ip route get 93.184.216.34
tracert example.com
mtr example.com
```

3 CHECK ARP / NEIGHBOR (Find the MAC)

```
ip neigh
arp -n
ping -c 1 93.184.216.34
ip neigh show 93.184.216.34
```

4 CAPTURE PACKETS (with tcpdump)

```
sudo tcpdump -i any \
  host example.com -w trace.pcap
```

5 GENERATE APPLICATION TRAFFIC

```
curl -v http://example.com
curl -v https://example.com
```

6 ANALYZE IN WIRESHARK

- Open **trace.pcap** in **Wireshark**
- Follow the TCP stream
- Inspect DNS, TCP handshake, HTTP/HTTPS request and response
- Look at timing, retransmissions, and any errors



KEY TAKEAWAY

Every application request goes through multiple network layers. Capturing and analyzing packets shows exactly what is happening.



REAL WORLD SKILL

This skill helps you troubleshoot slow applications, DNS issues, routing problems, and connectivity failures in production.



NEXT STEP

Repeat the lab with a failing scenario (e.g. blocked port, DNS failure, or high latency) and analyze the packet capture to find the root cause.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA

LEARN | BUILD | GROW

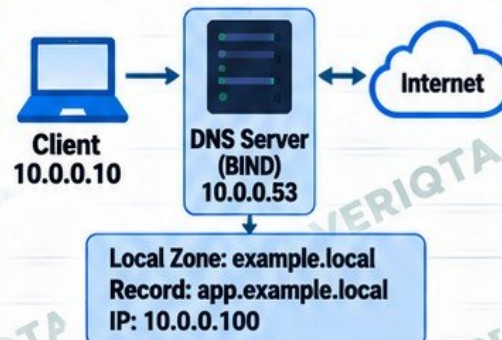
Build and Troubleshoot DNS

Run a local DNS environment. Create records, test recursive resolution, inspect TTL and caching, and diagnose NXDOMAIN, SERVFAIL, stale records, and resolver failures with **dig**.

PROJECT GOAL

- Set up a local DNS server (BIND).
- Create and manage DNS records.
- Test recursive and iterative resolution.
- Understand TTL and caching behavior.
- Diagnose common DNS failures.
- Use **dig** for investigation and troubleshooting.
- Apply real-world DNS best practices.

LAB TOPOLOGY



TOOLS USED

- BIND9 (DNS server)
- dig (DNS queries)
- nslookup (basic queries)
- tcpdump (packet capture)
- Wireshark (packet analysis)
- systemd (service management)
- named-checkconf (config check)
- rndc (DNS control)

DNS RECORD TYPES (COMMON)

- A - IPv4 address
- AAAA - IPv6 address
- CNAME - alias
- MX - mail server
- NS - name server
- TXT - text record

1. INSTALL AND CONFIGURE BIND9

```
# Ubuntu/Debian
sudo apt update
sudo apt install bind9 bind9utils

# Check service status
sudo systemctl status bind9
```

2. CREATE A DNS ZONE

```
# /etc/bind/named.conf.local
zone "example.local" {
    type master;
    file "/etc/bind/db.example.local";
};
```

3. ADD DNS RECORDS

```
# /etc/bind/db.example.local
$TTL 3600
@      IN SOA  ns1.example.local. (
    2024010101 3600 900 604800 86400 )
@      IN NS   ns1.example.local.
app    IN A    10.0.0.100
www    IN CNAME app.example.local.
```

4. TEST DNS RESOLUTION

```
# Query the local DNS server
dig @10.0.0.53 example.local

dig @10.0.0.53 app.example.local
dig app.example.local +short
```

5. CHECK TTL AND CACHING

```
# View TTL
dig @10.0.0.53 app.example.local

# Test caching behavior
dig @10.0.0.53 app.example.local
dig @10.0.0.53 app.example.local
+nocache
```

6. USE PACKET CAPTURE

```
# Capture DNS traffic
sudo tcpdump -i any port 53 -nn

# Open in Wireshark
sudo tcpdump -i any port 53 -w dns.pcap

# Then analyze dns.pcap in Wireshark
```

7. TROUBLESHOOT COMMON ISSUES

Issue	Command	What to Check
NXDOMAIN	dig bad.example.local	Typos, missing record
SERVFAIL	dig example.local	DNS server logs, zone file
Stale record	dig +nocache example.local	TTL, caching, reload zone
No response	dig @10.0.0.53 example.local	Firewall, service status
Wrong answer	dig example.local	Check config, recursion

8. USEFUL DEBUG COMMANDS

```
sudo named-checkconf
sudo named-checkzone example.local
/etc/bind/db.example.local
sudo rndc reload
sudo rndc status
dig +trace example.local
dig example.local @8.8.8.8
dig -x 10.0.0.100
```

9. CLEAN UP

```
sudo rndc stop
sudo systemctl stop bind9
sudo apt remove bind9
sudo rm -f /etc/bind/db.example.local
```



KEY TAKEAWAY

DNS issues can break entire applications. Always verify from the client side, the DNS server side, and the network path.



REAL WORLD SKILL

You will troubleshoot DNS issues in Linux, Docker, Kubernetes and cloud environments.



NEXT STEP

Extend this lab with recursive DNS, forwarders, DNS over TLS (DoT), and split-horizon DNS.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA
LEARN | BUILD | GROW

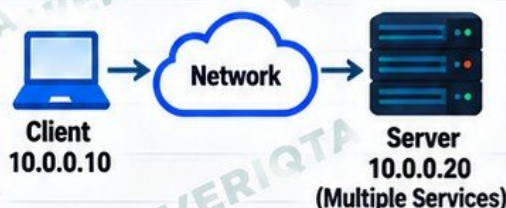
Diagnose TCP Connection Failures

Reproduce connection refused, timeout, reset, half-open connections, and port exhaustion. Analyze TCP states including SYN-SENT, ESTABLISHED, TIME-WAIT, and CLOSE-WAIT.

PROJECT GOAL

- Reproduce common TCP failures.
- Understand TCP states and transitions.
- Use real tools to diagnose issues.
- Analyze packet captures.
- Identify root cause from evidence.
- Apply fixes and verify.

LAB TOPOLOGY



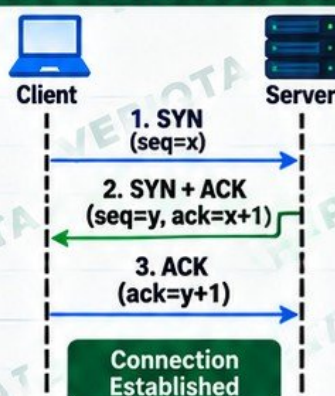
TOOLS USED

- **ip** (network info)
- **ss** (socket statistics)
- **netstat** (connections)
- **nc** (test connections)
- **tcpdump** (packet capture)
- **Wireshark** (analysis)
- **iptables** (simulate failures)
- **hping3** (advanced testing)
- **systemd** (manage services)

TCP CONNECTION FAILURE TYPES

Failure Type	What Happens	Common Cause
Connection Refused	Immediate RST	No service listening Port closed
Connection Timeout	No response	Firewall drop Network issue
Connection Reset	Connection closed	Service crash Firewall reset Application error
Half-Open Connection	Stuck in SYN-SENT	Server not responding Dropped SYN/ACK
Port Exhaustion	Cannot create new connections	Too many connections in TIME-WAIT

TCP 3-WAY HANDSHAKE



IMPORTANT TCP STATES

State	Meaning
SYN-SENT	SYN sent, waiting for SYN/ACK
SYN-RECEIVED	SYN received, waiting for ACK
ESTABLISHED	Connection active
FIN-WAIT-1	FIN sent, waiting for ACK
FIN-WAIT-2	ACK received, waiting for FIN
CLOSE-WAIT	Remote closed, waiting for local close
TIME-WAIT	Connection closed, waiting (2 × MSL)
LAST-ACK	FIN sent, waiting for ACK
CLOSED	No connection

1. CONNECTION REFUSED

```
# Try connecting to a closed port
nc -v 10.0.0.20 8080

# Check listening ports
ss -tulnp | grep 8080
```

2. CONNECTION TIMEOUT

```
# Simulate firewall drop
sudo iptables -A INPUT -p tcp \
--dport 8080 -j DROP

# Test connection
nc -v 10.0.0.20 8080
```

3. CONNECTION RESET

```
# Simulate reset
sudo iptables -A INPUT -p tcp \
--dport 8080 -j REJECT \
--reject-with tcp-reset

# Test connection
nc -v 10.0.0.20 8080
```

4. HALF-OPEN CONNECTION

```
# Send SYN packets only
sudo hping3 -S -p 8080 \
-c 5 10.0.0.20

# Check SYN-SENT state
ss -tan state syn-sent
```

5. PORT EXHAUSTION

```
# Create many connections
for i in {1..10000}; do
nc 10.0.0.20 8080 &
done

# Check connection count
ss -s
```

6. ANALYZE WITH TCPDUMP

```
# Capture packets
sudo tcpdump -i eth0 \
host 10.0.0.20 and tcp \
-w tcp-test.pcap

# Read output live
sudo tcpdump -i eth0 tcp
```

7. INSPECT IN WIRESHARK

- Open tcp-test.pcap
- Follow TCP stream
- Analyze flags (SYN, ACK, RST, FIN)
- Check timing between packets
- Identify where the failure occurs

8. VERIFY AND FIX

```
# Remove firewall rule
sudo iptables -D INPUT -p tcp \
--dport 8080 -j DROP

# Restart service if needed
sudo systemctl restart nginx

# Test again
nc -v 10.0.0.20 8080
```



KEY TAKEAWAY

Different failures look similar to users but have different causes. Use evidence (packets, states, logs) to find the real issue.



REAL WORLD SKILL

TCP issues are common in production. This project builds the skills to quickly diagnose and fix them.



NEXT STEP

Extend this lab with load balancers, Kubernetes services, and real application troubleshooting.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA
LEARN | BUILD | GROW

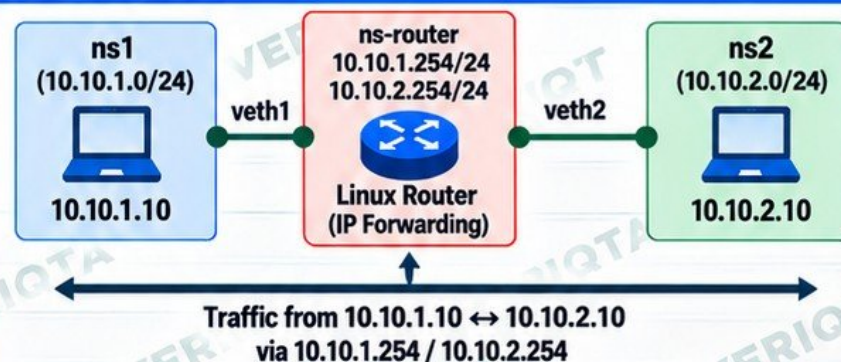
Build a Linux Routing Lab

Create multiple subnets and route traffic between them. Manipulate routing tables, default gateways, and static routes, then deliberately introduce asymmetric and missing-route failures.

PROJECT GOAL

- Create multiple isolated subnets using network namespaces.
- Configure routing between subnets.
- Work with default gateways and static routes.
- Test connectivity between hosts.
- Simulate and troubleshoot routing failures.
- Understand asymmetric routing and its impact.
- Use real Linux tools and commands.

LAB TOPOLOGY



IP ADDRESS PLAN

Node	Interface	IP Address	Subnet
ns1	veth1	10.10.1.10	10.10.1.0/24
ns-router	veth1	10.10.1.254	10.10.1.0/24
ns-router	veth2	10.10.2.254	10.10.2.0/24
ns2	veth2	10.10.2.10	10.10.2.0/24

TOOLS USED

- **ip** (network configuration)
- **ip route** (routing table)
- **ip netns** (network namespaces)
- **ping** (connectivity test)
- **traceroute** (path tracing)
- **tcpdump** (packet capture)
- **ss** (socket information)
- **nsenter** (enter namespace)

KEY CONCEPTS

- Network namespaces
- Virtual Ethernet (veth) pairs
- Routing tables
- Default gateways
- Static routes
- Longest prefix match
- Asymmetric routing
- Blackhole / missing routes
- Path verification with traceroute

1. CREATE NAMESPACES

```
sudo ip netns add ns1
ip netns add ns2
sudo ip netns add ns-router
```

2. CREATE VETH PAIRS

```
sudo ip link add veth1 type veth \
peer name veth1-r
sudo ip link add veth2 type veth \
peer name veth2-r
```

3. ASSIGN INTERFACES TO NAMESPACES

```
sudo ip link set veth1 netns ns1
sudo ip link set veth1-r netns ns-router
sudo ip link set veth2 netns ns2
sudo ip link set veth2-r netns ns-router
```

4. CONFIGURE IP ADDRESSES

```
# ns1
sudo ip netns exec ns1 ip addr add
10.10.1.10/24 dev veth1
sudo ip netns exec ns1 ip link set veth1 up
# ns-router
sudo ip netns exec ns-router ip addr add
10.10.1.254/24 dev veth1-r
sudo ip netns exec ns-router ip addr add
10.10.2.254/24 dev veth2-r
sudo ip netns exec ns-router ip link set veth1-r up
sudo ip netns exec ns-router ip link set veth2-r up
# ns2
sudo ip netns exec ns2 ip addr add 10.10.2.10/24 dev veth2
sudo ip netns exec ns2 ip link set veth2 up
```

5. CONFIGURE ROUTING

```
# Enable IP forwarding on router
sudo ip netns exec ns-router sysctl -w
net.ipv4.ip_forward=1
# Set default gateways
sudo ip netns exec ns1 ip route add
default via 10.10.1.254
sudo ip netns exec ns2 ip route add
default via 10.10.2.254
# Check routes
sudo ip netns exec ns1 ip route
sudo ip netns exec ns2 ip route
sudo ip netns exec ns-router ip route
```

6. TEST CONNECTIVITY

```
# From ns1 to ns2
sudo ip netns exec ns1 ping 10.10.2.10
sudo ip netns exec ns1 traceroute 10.10.2.10
# Capture traffic on router
sudo ip netns exec ns-router tcpdump -i any
```

7. INTRODUCE FAILURES (EXAMPLES)

- Remove a route: `ip route del ...`
- Wrong gateway: `ip route add default via 10.10.1.1`
- Asymmetric routing: only one side has a route
- Blackhole route: `ip route add 10.10.2.0/24 dev lo`
- MTU issue: `ip link set veth1 mtu 500`



KEY TAKEAWAY

Routing is about knowing the path, verifying the path, and understanding what happens when the path breaks.



REAL WORLD SKILL

These troubleshooting skills apply directly to servers, containers, Kubernetes and cloud networks.



NEXT STEP

Extend this lab with NAT, firewall rules and dynamic routing (e.g. FRR) in future projects.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta


VERIQTA
LEARN | BUILD | GROW

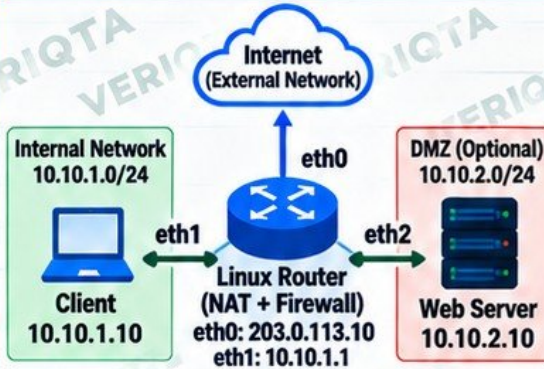
Build NAT and Firewall Rules

Configure forwarding, NAT, SNAT/DNAT, and packet filtering. Trace traffic through firewall rules and investigate why apparently valid connections fail.

PROJECT GOAL

- Enable IP forwarding on a Linux router.
- Configure NAT (SNAT and DNAT).
- Set up packet filtering with iptables (or nftables).
- Trace traffic through firewall rules.
- Simulate real-world scenarios (internet access, port forwarding).
- Investigate and fix connectivity failures.
- Understand how NAT and firewall rules work together.

LAB TOPOLOGY



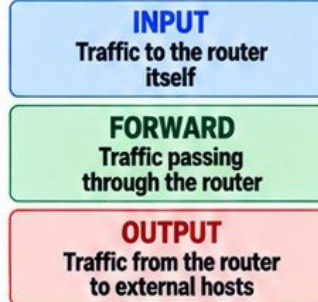
TOOLS USED

- iptables (firewall & NAT)
- nftables (alternative)
- ip (network commands)
- tcpdump (packet capture)
- curl / wget (testing)
- traceroute (path tracing)
- ping (connectivity test)
- ss (socket information)
- netstat (connection check)
- Wireshark (analysis)

NAT TYPES

Type	Purpose	Example Use Case
SNAT	Change source IP (private → public)	Allow internal hosts to access the internet
DNAT	Change destination IP (public → private)	Port forwarding to internal server
MASQUERADE	SNAT for dynamic public IP	Home/Cloud environments with dynamic IP
Static NAT	One-to-one IP mapping	Expose internal server with fixed public IP

FIREWALL RULE CHAINS



COMMON FIREWALL RULES

Rule	Purpose
ALLOW	Permit traffic
DROP	Silently drop traffic
REJECT	Drop and send an error
LOG	Log matching packets
STATEFUL	Allow established and related connections
DEFAULT DENY	Block all, then allow specific traffic

1. ENABLE IP FORWARDING

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | \
sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

2. CONFIGURE SNAT (MASQUERADE)

```
sudo iptables -t nat -A POSTROUTING \
-o eth0 -j MASQUERADE
```

3. CONFIGURE DNAT (PORT FORWARDING)

```
sudo iptables -t nat -A PREROUTING \
-i eth0 -p tcp --dport 80 \
-j DNAT --to-destination 10.10.2.10:80
sudo iptables -A FORWARD -p tcp \
-d 10.10.2.10 --dport 80 -j ACCEPT
```

4. BASIC FIREWALL RULES

```
# Allow established connections
sudo iptables -A FORWARD -m state \
--state ESTABLISHED,RELATED -j ACCEPT
# Allow internal to internet
sudo iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
# Default deny
sudo iptables -P FORWARD DROP
```

5. TRACE AND DEBUG

```
sudo tcpdump -i any
sudo iptables -L -v -n
sudo iptables -t nat -L -v -n
sudo traceroute 8.8.8.8
curl -v http://example.com
```

6. SIMULATE FAILURES

- Remove NAT rule and test internet access
- Block a port and observe connection failure
- Configure wrong DNAT IP and troubleshoot
- Drop return traffic and observe timeout
- Use logs to identify which rule is blocking
- Fix the issue and verify connectivity



KEY TAKEAWAY

NAT and firewall rules control how traffic flows. When a connection fails, always check the packet path, rule counters, and logs to find the real cause.



REAL WORLD SKILL

These skills are essential for Linux servers, cloud infrastructure, Kubernetes nodes, and production security troubleshooting.



NEXT STEP

Extend this lab with nftables, multiple WAN interfaces, and realistic DMZ architectures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA
LEARN | BUILD | GROW

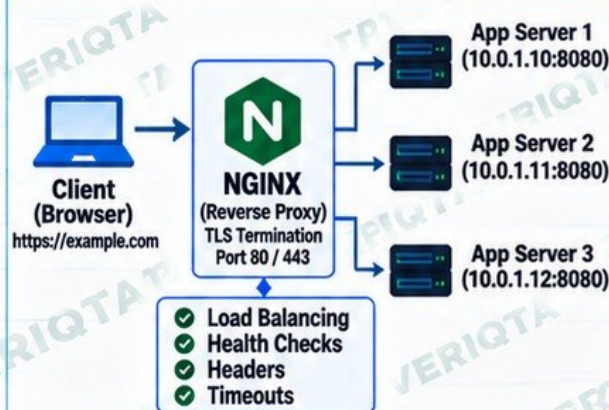
Build a Reverse Proxy with NGINX

Put multiple application servers behind NGINX. Configure upstreams, headers, timeouts, health behavior, and TLS termination. Diagnose 502, 503, and 504 failures.

PROJECT GOAL

- Install and configure NGINX as a reverse proxy.
- Set up multiple backend application servers.
- Configure upstreams, load balancing, headers, timeouts, and health checks.
- Enable TLS termination with HTTPS.
- Test and diagnose 502, 503, and 504 failures.
- Understand real-world reverse proxy best practices.

ARCHITECTURE DIAGRAM



TOOLS USED

- NGINX
- curl
- openssl
- nginx -t
- nginx -s reload
- tcpdump
- nginx logs
- browser / Postman
- (optional) Docker
- (optional) Certbot for TLS certificates

BASIC NGINX REVERSE PROXY CONFIG

```
upstream app_servers {
    server 10.0.1.10:8080;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080;
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://app_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }
}
```

KEY NGINX FEATURES

- Upstreams (multiple backend servers)
- Load balancing (round-robin, least_conn, etc.)
- Health checks (detect unhealthy backends)
- Header management (X-Forwarded-For, etc.)
- Timeouts (connect, send, read)
- TLS termination (HTTPS)
- Caching (optional)
- Compression (optional)
- Logging and monitoring

COMMON HTTP ERRORS

Error	Meaning
502 Bad Gateway	NGINX cannot reach the backend server.
503 Service Unavailable	No healthy backends available or all servers down.
504 Gateway Timeout	Backend took too long to respond (timeout).

TESTING COMMANDS

```
# Test the reverse proxy
curl -I http://example.com
curl -I https://example.com

# Check backend response
curl -I http://10.0.1.10:8080

# Test with verbose output
curl -v http://example.com

# Check NGINX configuration
sudo nginx -t
sudo nginx -s reload
```

TROUBLESHOOTING

- Check NGINX error logs (/var/log/nginx/error.log)
- Verify backend servers are running.
- Test backend connectivity from NGINX host.
- Check firewall and security group rules.
- Validate upstream configuration.
- Inspect timeout settings.
- Use curl and tcpdump to trace traffic.
- Check application logs on backend servers.
- Verify SSL/TLS certificates (if using HTTPS).

REAL-WORLD SCENARIOS

- Load balance multiple application instances.
- Handle backend server failure gracefully.
- Use HTTPS with SSL certificates.
- Troubleshoot intermittent 502/503/504 errors.
- Configure custom headers for applications.
- Deploy in containers or cloud environments.



KEY TAKEAWAY

A reverse proxy improves security, reliability, and performance. When things fail, always check the full request path: client → NGINX → backend → application.



NEXT STEP

Extend this project with Docker, automated TLS (Certbot), monitoring (Prometheus + Grafana), and deployment on Kubernetes (Ingress).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA
LEARN | BUILD | GROW

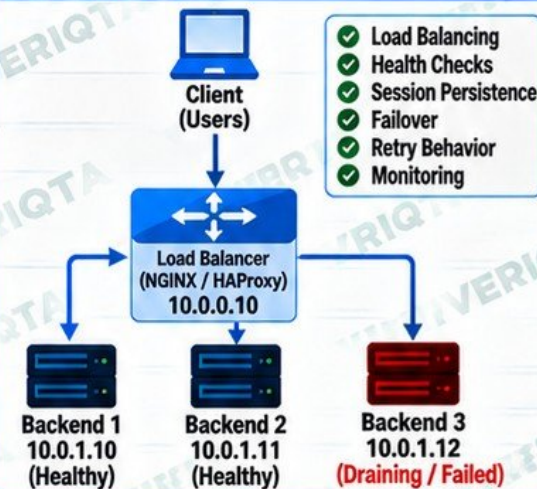
Build a Production Load-Balancing Lab

Deploy multiple backend instances behind a load balancer. Test health checks, connection distribution, backend failure, draining, session persistence, and retry behavior.

PROJECT GOAL

- Set up a realistic load-balancing environment.
- Deploy multiple backend instances (e.g. web servers).
- Configure health checks.
- Test connection distribution.
- Simulate backend failure and observe behavior.
- Test draining and graceful shutdown.
- Implement session persistence.
- Configure and test retry behavior.
- Understand real-world load balancing best practices.

LAB TOPOLOGY



TOOLS USED

- NGINX (load balancer)
- HAProxy (alternative)
- Docker (backend instances)
- curl (testing)
- ab / wrk (load testing)
- htop (monitoring)
- tcpdump (traffic analysis)
- nginx / haproxy logs
- Prometheus + Grafana (optional)
- Linux networking tools (ip, ss, netstat)

EXAMPLE NGINX CONFIG (LOAD BALANCING)

```
upstream app_servers {
    server 10.0.1.10:8080;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080;
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://app_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_next_upstream error timeout http_502 http_503 http_504;
    }
}
```

LOAD BALANCING METHODS

- Round Robin
- Least Connections
- IP Hash (Session Persistence)
- Weighted (Different server capacity)
- Least Response Time
- Random
- Custom (e.g. based on headers)

HEALTH CHECK EXAMPLE

```
location /health {
    access_log off;
    return 200 "healthy\n";
}
```

TESTING COMMANDS

```
# Test load balancing
for i in {1..10}; do
    curl http://10.0.0.10;
done

# Load test
wrk -t4 -c100 -d30s http://10.0.0.10

# Check backend health
curl http://10.0.1.10/health

# Simulate backend failure
docker stop backend3

# Monitor logs
tail -f /var/log/nginx/access.log
```

SCENARIOS TO TEST

- 1 Normal traffic distribution
- 2 Backend instance failure
- 3 Health check detection
- 4 Connection draining (graceful shutdown)
- 5 Session persistence (IP hash or cookie)
- 6 Retry behavior on failure
- 7 Overloaded backend
- 8 Recovering a failed backend



KEY TAKEAWAY

- ✓ Load balancers improve availability, scalability, and performance.
- ✓ Health checks prevent traffic from going to unhealthy instances.
- ✓ Draining allows graceful maintenance.
- ✓ Session persistence is important for stateful applications.
- ✓ Retry behavior helps with temporary failures.
- ✓ These skills are essential in real production environments.



NEXT STEP

- Try HAProxy instead of NGINX.
- Add TLS termination (HTTPS).
- Use Docker Compose or Kubernetes.
- Integrate monitoring (Prometheus + Grafana).
- Test with real applications (e.g. WordPress, Node.js, Python).
- Experiment with advanced load balancing features (weighted, sticky sessions).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA
LEARN | BUILD | GROW

Build and Debug TLS

Create certificates, configure HTTPS, inspect the TLS handshake, certificate chain, SANs, expiration, and trust. Reproduce hostname, CA, and expired-certificate failures using openssl s_client.

PROJECT GOAL

- Understand how TLS works in practice.
- Create and manage certificates.
- Configure HTTPS on a web server (NGINX).
- Inspect the TLS handshake and certificate details.
- Verify certificate chain, SANs and expiration.
- Reproduce and troubleshoot common TLS failures.
- Use openssl s_client for deep analysis.
- Apply real-world best practices.

TLS HANDSHAKE FLOW



Client

Web Server
HTTPS (TLS)

- 1 ClientHello (TLS version, cipher suites, SNI)
- 2 ServerHello (selected cipher, certificate)
- 3 Certificate (server cert + chain)
- 4 Key Exchange (establish session keys)
- 5 Finished (secure communication)

TOOLS USED

- openssl (s_client, x509)
- NGINX / Apache
- curl (testing HTTPS)
- keytool (view keystore)
- certbot (Let's Encrypt)
- Wireshark (TLS analysis)
- ss (check open ports)
- dig (DNS and hostname)
- python -m http.server (simple test server)

CREATE A SELF-SIGNED CERTIFICATE

```
# Generate private key
openssl genrsa -out server.key 2048

# Create self-signed certificate
openssl req -x509 -new -nodes -key server.key \
  -sha256 -days 365 -out server.crt \
  -subj "/CN=example.com"

# View certificate details
openssl x509 -in server.crt -text -noout
```

CONFIGURE NGINX WITH HTTPS

```
server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /etc/nginx/server.crt;
    ssl_certificate_key /etc/nginx/server.key;

    location / {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
    }
}
```

INSPECT TLS WITH OPENSSSL

```
# Test TLS connection
openssl s_client \
  -connect example.com:443 \
  -servername example.com

# Show certificate chain
openssl s_client -connect \
  example.com:443 -showcerts

# Check specific certificate fields
openssl x509 -in server.crt -text -noout

# Check expiry date
openssl x509 -in server.crt -noout -dates
```

COMMON TLS FAILURES AND HOW TO REPRODUCE

Failure Type	How to Reproduce	Example Command
Hostname mismatch	Use wrong hostname	openssl s_client -connect 1.2.3.4:443 \ -servername wrong.com
Untrusted CA	Use self-signed cert	curl https://example.com # shows certificate error
Expired certificate	Use an expired cert	openssl s_client -connect \ expired.com:443
Missing intermediate CA	Configure incomplete chain	openssl s_client -connect \ example.com:443 -showcerts
Wrong port / no TLS	Connect to non-TLS port	openssl s_client -connect \ example.com:80
TLS version/cipher issue	Force old TLS version	openssl s_client -tls1 \ -connect example.com:443

CERTIFICATE DETAILS TO CHECK

- ✓ Common Name (CN)
- ✓ Subject Alternative Names (SANs)
- ✓ Issuer (CA)
- ✓ Validity period (Not Before / Not After)
- ✓ Certificate chain (intermediate CAs)
- ✓ Signature algorithm
- ✓ Key size and type (RSA/ECDSA)
- ✓ Revocation status (OCSP/CRL)

TEST HTTPS WITH CURL

```
# Basic test
curl -v https://example.com

# Show certificate details
curl -vI https://example.com

# Test with specific TLS version
curl --tls1.2 https://example.com
```



KEY TAKEAWAY

- ✓ TLS ensures encrypted and trusted communication.
- ✓ Always verify the full certificate
- ✓ Check SANs, expiration, and hostname match.
- ✓ Use openssl s_client for deep troubleshooting.
- ✓ Understand common failure scenarios and fixes.



NEXT STEP

- Automate certificate management with Certbot.
- Implement TLS monitoring and alerts.
- Explore mutual TLS (mTLS).
- Use HTTPS in real applications (Docker, Kubernetes, cloud).
- Practice with Wireshark to analyze TLS handshakes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA
LEARN | BUILD | GROW

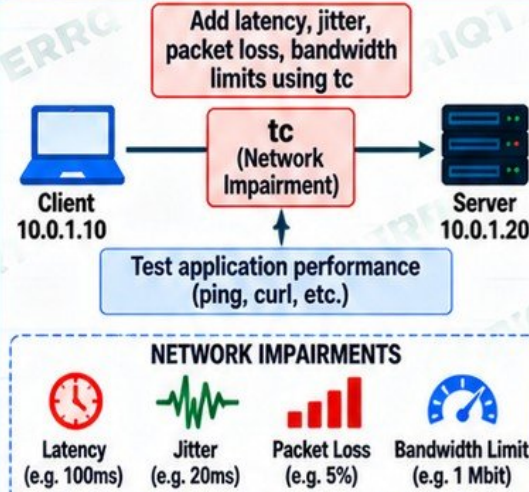
Diagnose Latency and Packet Loss

Introduce latency, jitter, packet loss, and bandwidth constraints with Linux tc. Determine whether degraded application performance originates from compute, network, or dependencies.

PROJECT GOAL

- Use Linux tc to simulate real-world network conditions.
- Introduce latency, jitter, packet loss, and bandwidth limits.
- Measure and analyze the impact on applications.
- Determine if performance issues originate from compute, network, or dependencies.
- Use monitoring and troubleshooting tools.
- Apply findings to real-world scenarios.

LAB TOPOLOGY



TOOLS USED

- tc (traffic control)
- ip (network commands)
- ping
- mtr (path analysis)
- iperf3 (bandwidth testing)
- curl / wget (application test)
- ss (socket information)
- netstat (connection check)
- htop (system monitoring)
- tcpdump (packet capture)
- Wireshark (analysis)
- Grafana + Prometheus (optional)

INTRODUCE NETWORK IMPAIRMENTS

```
# Add 100ms latency
sudo tc qdisc add dev eth0 root netem delay 100ms

# Add latency with jitter
sudo tc qdisc add dev eth0 root netem delay 100ms 20ms

# Add 5% packet loss
sudo tc qdisc add dev eth0 root netem loss 5%

# Limit bandwidth to 1 Mbit
sudo tc qdisc add dev eth0 root tbf rate 1mbit burst 32kbit latency 400ms

# Remove all rules
sudo tc qdisc del dev eth0 root
```

TEST AND MEASURE

```
# Test latency and packet loss
ping -c 20 10.0.1.20

# Continuous ping with stats
ping -i 0.2 10.0.1.20

# Path analysis
mtr 10.0.1.20

# Bandwidth test
iperf3 -c 10.0.1.20

# Test application (HTTP)
curl -w "time_total: %{time_total}\n" -o /dev/null -s http://10.0.1.20

# Capture packets
sudo tcpdump -i eth0 -w test.pcap
```

ANALYZE RESULTS

- Check latency (avg, min, max)
- Measure packet loss percentage
- Identify jitter (variation in latency)
- Compare application response times before and after impairment
- Use tcpdump/Wireshark to verify packet behavior
- Check server resource usage (CPU, memory, disk)
- Determine if the issue is network, compute, or a dependency (database, external API, etc.)

EXAMPLE SCENARIOS

- 1 Add 200ms latency and test web app response.
- 2 Introduce 10% packet loss and observe retries.
- 3 Limit bandwidth to 512 Kbit and test file download.
- 4 Combine latency + jitter + packet loss.
- 5 Diagnose whether slowness is network or server side.
- 6 Test impact on database connection.
- 7 Verify recovery after removing tc rules.



KEY TAKEAWAY

- ✓ Network issues can severely impact application performance.
- ✓ Use tc to simulate real-world conditions.
- ✓ Measure and analyze before assuming the cause.
- ✓ Always verify if the issue is network, compute, or a dependency.
- ✓ These skills are essential for production troubleshooting.



NEXT STEP

- Integrate with monitoring (Prometheus + Grafana).
- Create automated network testing scripts.
- Test with real applications (Nginx, database, microservices).
- Add this lab to a Kubernetes environment using network policies.
- Document findings and build runbooks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA

LEARN | BUILD | GROW

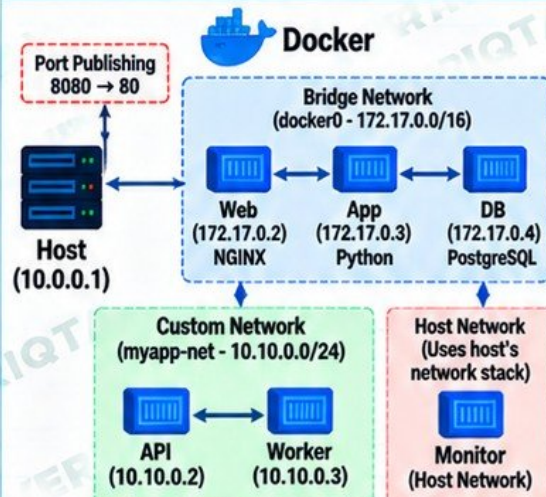
Build Docker Container Networking

Compare bridge, host, and custom networks. Inspect namespaces, virtual Ethernet interfaces, container DNS, port publishing, and host-to-container traffic paths.

PROJECT GOAL

- Understand Docker networking drivers (bridge, host, custom).
- Inspect network namespaces and virtual Ethernet (veth) pairs.
- Test container DNS and service discovery.
- Configure and test port publishing (host ↔ container).
- Analyze host-to-container and container-to-container traffic paths.
- Troubleshoot common networking issues.
- Apply best practices for real-world container environments.

NETWORK TOPOLOGY EXAMPLE



TOOLS USED

- docker (CLI)
- ip (network namespaces)
- nsenter
- docker network
- docker inspect
- docker run
- tcpdump (traffic capture)
- curl / wget (testing)
- dig (DNS testing)
- ss / netstat (connections)
- wireshark (optional)
- iptables (understanding)

DOCKER NETWORK DRIVERS

Driver	Description	Use Case
bridge	Default network driver. Containers on isolated network (NAT).	Most common (local development).
host	Container uses host network stack.	High performance, monitoring tools.
none	No networking (fully isolated).	Security, custom networking.
overlay	Multi-host networking (for Swarm).	Distributed apps (across multiple hosts).
macvlan	Container gets its own MAC address on the LAN.	Advanced networking (use with caution).
custom	User-defined bridge network.	Better control, DNS, multi-container apps.

USEFUL COMMANDS

```
# List networks
docker network ls

# Inspect a network
docker network inspect bridge

# Create a custom network
docker network create myapp-net

# Run a container on custom network
docker run -d --name api \\\
--network myapp-net nginx

# Publish port (host:container)
docker run -d -p 8080:80 nginx

# Inspect container network details
docker inspect <container_id>

# Enter container namespace
nsenter -t $(docker inspect -f '{{.State.Pid}}' \\\
<container_id>) -n ip addr

# Test DNS inside container
docker exec -it <container> dig db

# Capture traffic on docker0
tcpdump -i docker0 -nn
```

CONTAINER DNS

- Each container has an internal DNS resolver (127.0.0.11).
- Containers on the same custom network can resolve each other by name.
- Use --network and container names for service discovery.

```
# Test DNS resolution
docker exec -it api dig db

# See DNS configuration
docker exec -it api cat /etc/resolv.conf
```

INSPECT NETWORK NAMESPACES

```
# Get container PID
docker inspect -f '{{.State.Pid}}' <container>

# Enter the network namespace
nsenter -t <PID> -n ip addr

# View veth pairs on host
ip link | grep veth
```

TEST SCENARIOS

- 1 Deploy containers on bridge, host, and custom networks.
- 2 Test container-to-container communication.
- 3 Test host-to-container and container-to-host traffic.
- 4 Verify DNS resolution between containers.
- 5 Inspect virtual Ethernet interfaces and namespaces.
- 6 Capture and analyze network traffic with tcpdump.
- 7 Simulate and troubleshoot networking issues (e.g. DNS failure, blocked ports).



KEY TAKEAWAY

- ✓ Docker networking provides flexible options for different use cases.
- ✓ Understand how containers communicate with each other and the host.
- ✓ Custom networks improve isolation and service discovery.
- ✓ Inspect network namespaces and veth pairs to troubleshoot.
- ✓ Mastering container networking is essential for real-world DevOps and Kubernetes.



NEXT STEP

- Explore Docker Compose for multi-container applications.
- Test overlay networks (Docker Swarm).
- Integrate with Kubernetes networking (CNI).
- Implement network policies and service mesh (e.g. Istio).
- Practice more real-world scenarios (e.g. microservices, database, monitoring).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA
LEARN | BUILD | GROW

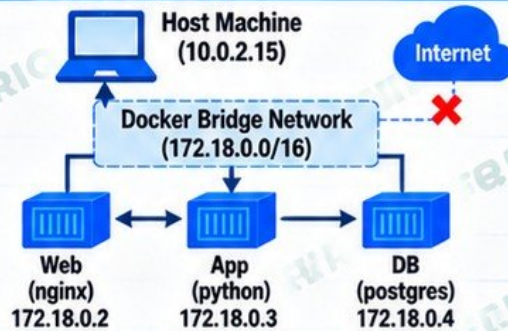
Troubleshoot Docker Networking Failures

Break DNS, published ports, routes, container-to-container communication, and host connectivity. Investigate each failure from the packet path rather than restarting containers blindly.

PROJECT GOAL

- Reproduce common Docker networking failures.
- Break and fix DNS resolution.
- Troubleshoot published ports (host ↔ container).
- Diagnose container-to-container communication issues.
- Investigate routing and connectivity problems.
- Use packet path analysis instead of blind container restarts.
- Apply systematic troubleshooting steps and best practices.

LAB ENVIRONMENT



Common Failure Points

- ✗ DNS resolution
- ✗ Published ports (8080:80)
- ✗ Container-to-container traffic
- ✗ Routes / default gateway
- ✗ Host connectivity

TOOLS USED

- docker (CLI)
- docker network
- docker inspect
- docker logs
- docker exec
- ip / iptables
- nsenter
- ping
- curl / wget
- dig / nslookup
- tcpdump
- ss / netstat
- traceroute / mtr
- iptables / nftables
- wireshark (optional)
- netshoot (troubleshooting image)

COMMON FAILURES AND HOW TO REPRODUCE

Failure Type	How to Reproduce	Expected Issue
DNS failure	Use an invalid DNS or block DNS port 53	Containers cannot resolve names
Published port not working	Run container without -p or wrong port	Host cannot reach container service
Container-to-container failure	Use different networks or wrong service name	Containers cannot communicate
Routing failure	Remove default gateway or wrong route	No external connectivity
Host connectivity failure	Block traffic with iptables	Host cannot reach container or vice versa
External service failure	Block outbound traffic (DNS/HTTP)	Cannot reach external APIs

USEFUL TROUBLESHOOTING COMMANDS

```
# Check container networks
docker network ls
docker network inspect bridge

# Check container details
docker inspect <container>

# Test DNS inside container
docker exec -it <container> dig google.com

# Test connectivity
ping <ip or hostname>
curl -v http://<service>

# Check listening ports
ss -tlnp | grep <port>

# Capture packets
tcpdump -i any -nn host <ip>

# Check iptables rules
sudo iptables -L -n -v

# Enter network namespace
sudo nsenter -t $(docker inspect -f '{{.State.Pid}}' <container>) -n ip addr
```

PACKET PATH ANALYSIS

- 1 Client (host or container) sends a request.
- 2 Check DNS resolution (name → IP).
- 3 Check routing (which interface and gateway).
- 4 Check firewall rules (iptables).
- 5 Check container network (bridge, overlay, custom).
- 6 Verify the service is listening.
- 7 Capture and analyze packets (tcpdump / Wireshark).
- 8 Identify where the connection is failing.

REAL-WORLD SCENARIOS

- 1 Container cannot resolve database hostname.
- 2 Host cannot access published port (e.g. localhost:8080).
- 3 Containers on different networks cannot communicate.
- 4 No internet access from container.
- 5 Intermittent connectivity or dropped packets.
- 6 Firewall blocking traffic between containers.
- 7 External API calls failing from container.



KEY TAKEAWAY

- ✓ Troubleshoot the packet path, not just the container.
- ✓ Check DNS, routing, firewall, and service configuration.
- ✓ Use the right tools to gather evidence (tcpdump, inspect, logs).
- ✓ Understand Docker networking drivers and namespaces.
- ✓ Systematic troubleshooting saves time and prevents repeat issues.



NEXT STEP

- Create advanced scenarios with multiple networks.
- Integrate with Kubernetes (CNI comparison).
- Test with real applications (e.g. web app + database).
- Build a troubleshooting runbook.
- Explore network policies and service mesh (e.g. Istio).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA
LEARN | BUILD | GROW

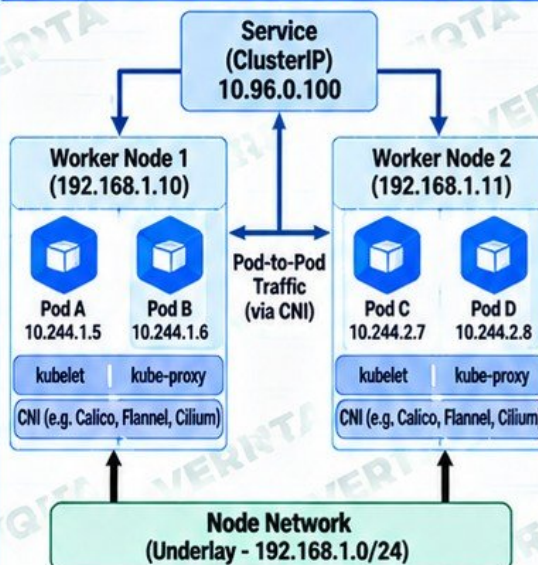
Trace Kubernetes Pod-to-Pod Networking

Deploy workloads across nodes and trace Pod traffic through the CNI network. Examine Pod IPs, routes, namespaces, Services, endpoints, and node-level forwarding.

PROJECT GOAL

- Deploy workloads across multiple nodes.
- Understand how Pod-to-Pod networking works in Kubernetes.
- Identify Pod IPs, node IPs, and CNI network configuration.
- Trace traffic between Pods across nodes.
- Examine Services, endpoints, and kube-proxy behavior.
- Inspect network namespaces and virtual interfaces.
- Verify node-level forwarding and routing.
- Troubleshoot common connectivity issues.
- Apply real-world debugging techniques.

KUBERNETES NETWORKING TOPOLOGY



TOOLS USED

- kubectl
- ip (iproute2)
- nsenter
- tcpdump
- wireshark (optional)
- cni plugins (Calico/Flannel/Cilium)
- helm (optional)
- jq (parsing)
- kubectl sniff (optional)
- kubectl debug
- iptables / nftables
- ipvsadm (if using IPVS)
- dig / nslookup
- curl / wget
- traceroute / mtr
- netstat / ss

USEFUL KUBECTL COMMANDS

```
# Get nodes and Pod IPs
kubectl get nodes -o wide
kubectl get pods -o wide -A

# Get Services and endpoints
kubectl get svc -A
kubectl get endpoints -A

# Describe a Pod
kubectl describe pod <pod-name>

# Exec into a Pod
kubectl exec -it <pod-name> - sh

# Run a temporary debug Pod
kubectl run netshoot --rm -it --image=nicolaka/netshoot -- sh
```

NODE AND NAMESPACE INSPECTION

```
# Check network namespaces
sudo ip netns list
sudo nsenter -t <pid> -n ip addr

# Check routes
ip route

# Check CNI interfaces
ip link | grep -E "cni|flannel|calico|cilium"

# Check iptables / IPVS rules
sudo iptables -t nat -L -n
sudo ipvsadm -L -n

# Capture Pod-to-Pod traffic
sudo tcpdump -i any host <pod-ip>

# Test connectivity
ping <pod-ip>
curl http://<pod-ip>:<port>
```

TRACE A POD-TO-POD CONNECTION

- 1 Identify source and destination Pods.
- 2 Check Pod IPs and node location.
- 3 Verify routing between nodes.
- 4 Inspect CNI network configuration.
- 5 Use tcpdump on source, destination, and node interfaces.
- 6 Trace through kube-proxy (iptables/IPVS).
- 7 Verify NetworkPolicies (if enabled).
- 8 Identify where the traffic is dropped (if any).

COMMON ISSUES AND HOW TO FIX

Issue	Possible Cause	Fix
Pods cannot reach each other	CNI misconfiguration or NetworkPolicy	Check CNI, routes, NetworkPolicies
Pod can reach local Pod but not remote Pod	Node routing or encapsulation issue	Verify node routes, CNI overlay settings
DNS not working	CoreDNS issue	Check CoreDNS Pods and ConfigMap
Service not reachable	kube-proxy / iptables issue	Check iptables/IPVS rules
Intermittent connectivity	NetworkPolicy or MTU issue	Review policies, check MTU settings



KEY TAKEAWAY

- ✓ Pod-to-Pod communication is handled by the CNI plugin.
- ✓ Understand the full packet path (source Pod → CNI → node → CNI → destination Pod).
- ✓ Use the right tools to gather evidence before making changes.
- ✓ Check Services, endpoints, iptables/IPVS, and NetworkPolicies.
- ✓ Systematic tracing helps you find the real issue faster.



NEXT STEP

- Try with different CNI plugins (Calico, Flannel, Cilium).
- Implement and test NetworkPolicies.
- Explore advanced topics: service mesh (Istio), eBPF networking (Cilium), and DNS policies.
- Build a troubleshooting runbook for Kubernetes networking.
- Practice real-world scenarios (e.g. multi-node cluster, cross-namespace communication, policy-based blocking).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA
LEARN | BUILD | GROW

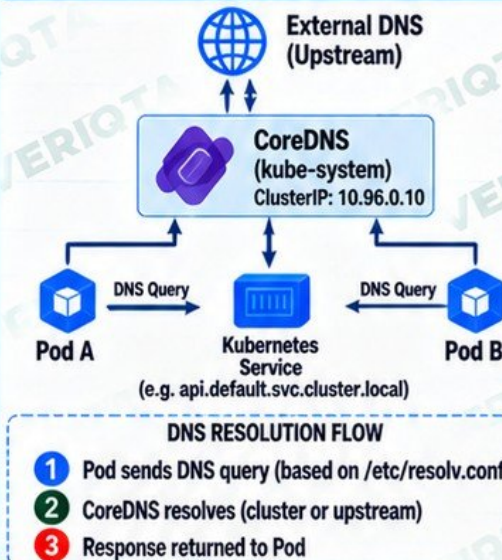
Troubleshoot Kubernetes DNS

Build a DNS incident lab around CoreDNS. Diagnose failed service discovery, incorrect search domains, ndots, resolver configuration, upstream DNS failures, and DNS latency.

PROJECT GOAL

- Understand how DNS works in Kubernetes (CoreDNS).
- Reproduce and troubleshoot common DNS failures.
- Inspect search domains, ndots, and resolver configuration.
- Diagnose failed service discovery (both internal and external).
- Simulate upstream DNS failures and DNS latency.
- Use the right tools to trace DNS queries end-to-end.
- Apply real-world troubleshooting methodology.

KUBERNETES DNS ARCHITECTURE



TOOLS USED

- kubectl
- dig
- nslookup
- kubectl exec
- kubectl logs
- kubectl get endpoints
- kubectl describe svc
- kubectl get cm (CoreDNS)
- tcpdump
- k9s (optional)
- stern (optional)
- prometheus + grafana (optional for latency)
- dnsutils (in debug pod)
- busybox / alpine (test pod)

USEFUL KUBECTL COMMANDS

```
# Check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns

# View CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns

# Get CoreDNS ConfigMap
kubectl get cm coredns -n kube-system -o yaml

# Describe a service
kubectl describe svc <service-name>

# Get endpoints
kubectl get endpoints <service-name>

# Check Pod DNS configuration
kubectl exec -it <pod> -- cat /etc/resolv.conf

# Run a DNS query from a Pod
kubectl exec -it <pod> -- nslookup <service-name>
```

TEST DNS RESOLUTION

```
# Query a Kubernetes service
dig myservice.default.svc.cluster.local

# Query with short name (uses search domains)
dig myservice

# Query external domain
dig kubernetes.io

# Use nslookup
nslookup myservice.default.svc.cluster.local

# Check DNS latency
dig +stats kubernetes.io

# Test with different ndots
dig @10.96.0.10 myservice

# Continuous DNS query (latency test)
while true; do dig +time=1 kubernetes.io; sleep 1; done
```

COMMON DNS FAILURES

- 1 Service name not resolved (NXDOMAIN)
- 2 Incorrect search domains
- 3 ndots causing unexpected lookups
- 4 CoreDNS pod failure or crash loop
- 5 Upstream DNS server unreachable
- 6 High DNS latency / timeouts
- 7 DNS works for some pods but not others
- 8 External domain resolution failing
- 9 Intermittent DNS failures

EXAMPLE SCENARIOS

- 1 Reproduce a service discovery failure (NXDOMAIN).
- 2 Misconfigure ndots and observe query behavior.
- 3 Break upstream DNS and test external resolution.
- 4 Introduce DNS latency using tc.
- 5 Test DNS from different namespaces.
- 6 Compare short name vs FQDN resolution.
- 7 Analyze CoreDNS logs and metrics.
- 8 Restore configuration and verify successful resolution.



KEY TAKEAWAY

- ✓ Kubernetes uses CoreDNS for cluster DNS (service discovery and external resolution).
- ✓ Understand search domains and ndots.
- ✓ Always check /etc/resolv.conf inside the Pod.
- ✓ Use the right tools (dig, nslookup, tcpdump) to trace DNS queries.
- ✓ DNS latency and upstream failures can seriously impact applications.
- ✓ Systematic troubleshooting saves time and prevents unnecessary restarts.



NEXT STEP

- Monitor CoreDNS with Prometheus and Grafana.
- Set up alerts for DNS failures and high latency.
- Explore advanced topics: stubDomains, NodeLocal DNSCache, and custom DNS policies.
- Integrate with real deployments (e.g. multi-cluster, hybrid cloud).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA
LEARN | BUILD | GROW

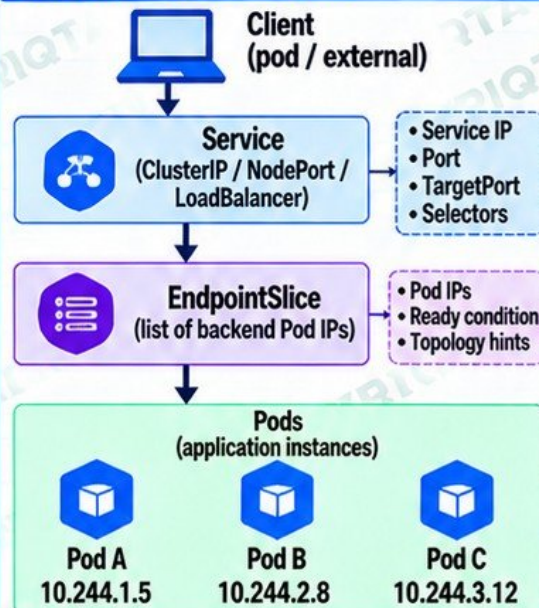
Trace Kubernetes Service Traffic

Follow Client → Service → EndpointSlice → Pod. Investigate selectors, ports, target ports, readiness, kube-proxy or equivalent service-routing behavior, and missing endpoints.

PROJECT GOAL

- Understand how Kubernetes Services route traffic to Pods.
- Trace the full path: Client → Service → EndpointSlice → Pod.
- Inspect selectors, ports, and target ports.
- Verify Pod readiness and how it affects endpoints.
- Understand kube-proxy (iptables/IPVS) or equivalent service-routing behavior.
- Troubleshoot missing or incorrect endpoints.
- Use practical tools to trace and analyze service traffic.
- Apply real-world troubleshooting methodology.

SERVICE TRAFFIC FLOW



TOOLS USED

- kubectl
- kubectl get svc
- kubectl get endpointslices
- kubectl get endpoints
- kubectl describe svc
- kubectl describe endpointslices
- kubectl get pods
- kubectl describe pod
- kubectl logs
- kubectl exec
- curl / wget
- nslookup / dig
- tcpdump
- iptables / ipvsadm
- k9s (optional)
- stern (optional)
- Wireshark (optional)
- netshoot (troubleshooting pod)
- prometheus + grafana (optional for metrics)

USEFUL KUBECTL COMMANDS

```

# List services
kubectl get svc -A

# Describe a service
kubectl describe svc <service-name>

# Get endpoints and EndpointSlices
kubectl get endpoints -n <namespace>
kubectl get endpointslices -n <namespace>

# Describe EndpointSlice
kubectl describe endpointslice <name> -n <namespace>

# Check pod readiness
kubectl get pods -o wide -n <namespace>
kubectl describe pod <pod-name> -n <namespace>

# Exec into a pod and test connectivity
kubectl exec -it <pod> -- curl -v <service-name>:<port>

# Check kube-proxy mode
kubectl get cm kube-proxy -n kube-system -o yaml

```

INSPECT SERVICE CONFIGURATION

```

# View service YAML
kubectl get svc <service-name> -o yaml

# Check selectors
kubectl get svc <service-name> -o yaml | grep selector

# Verify endpoints match selectors
kubectl get endpoints <service-name> -o wide

# Check EndpointSlice details
kubectl get endpointslices -l
kubernetes.io/service-name=<service-name> -o wide

# Check target ports and pod container ports
kubectl describe pod <pod-name> | grep -A 5 Ports

# Test service from another pod
kubectl run test --rm -it --image=busybox --
sh -c "wget -O- <service-name>:<port>"

```

COMMON ISSUES AND CHECKS

- 1 Service has no endpoints (selector mismatch).
- 2 Pods are not ready (readiness probe failing).
- 3 Wrong targetPort or port configuration.
- 4 EndpointSlice not created or outdated.
- 5 kube-proxy rules not updated (iptables/IPVS).
- 6 NetworkPolicy blocking traffic.
- 7 DNS resolution issue for service name.
- 8 Pod crashes or restarts.
- 9 Traffic drops due to node/network issues.
- 10 Service type misconfiguration (ClusterIP/NodePort/LoadBalancer).

EXAMPLE SCENARIOS

- 1 Service has no endpoints and returns "connection refused".
- 2 Selector mismatch (labels don't match Pods).
- 3 Pod not ready (readiness probe failing).
- 4 Wrong targetPort causing 502/503 errors.
- 5 Trace traffic from client to Pod using tcpdump.
- 6 Verify kube-proxy routing rules (iptables or IPVS).
- 7 NetworkPolicy blocking service traffic.
- 8 Debug intermittent service connectivity.

KEY TAKEAWAY

- ✓ Kubernetes Services use selectors to find ready Pods via EndpointSlices.
- ✓ Always verify selectors, ports, and targetPorts.
- ✓ No endpoints = no traffic.
- ✓ Readiness controls whether a Pod receives traffic.
- ✓ kube-proxy or equivalent programs service routing (iptables/IPVS/eBPF).
- ✓ Use the right tools to trace traffic end-to-end.
- ✓ Systematic troubleshooting is faster and more effective than restarting Pods.

NEXT STEP

- Explore different service types (ClusterIP, NodePort, LoadBalancer).
- Investigate headless services.
- Test with real applications (e.g. Nginx, microservices).
- Combine with NetworkPolicies.
- Monitor service metrics with Prometheus.
- Build more complex scenarios (multi-namespace, cross-cluster).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta


VERIQTA
LEARN | BUILD | GROW

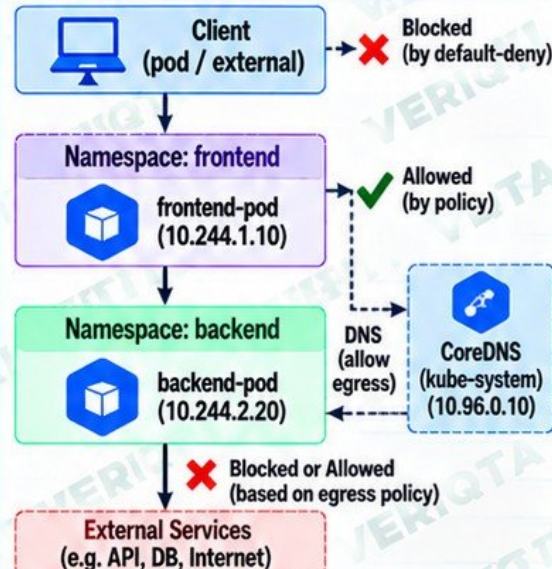
Build Kubernetes Network Segmentation

Implement default-deny and allow-list NetworkPolicies. Test ingress and egress controls, DNS dependencies, namespace isolation, and the limitations imposed by the chosen CNI implementation.

PROJECT GOAL

- Understand Kubernetes network segmentation and zero-trust principles.
- Implement default-deny NetworkPolicies.
- Create allow-list policies for specific ingress and egress traffic.
- Test DNS dependencies (CoreDNS).
- Validate namespace isolation.
- Observe the behavior and limitations of different CNI implementations (Calico, Cilium, Flannel, etc.).
- Troubleshoot common NetworkPolicy issues.
- Apply real-world best practices for secure cluster networking.

LAB ARCHITECTURE



TOOLS USED

- kubectl
- k9s (optional)
- kubectl explain
- kubectl get / describe
- kubectl apply
- kubectl exec
- kubectl logs
- curl / wget
- nslookup / dig
- tcpdump
- helm (optional)
- Calico / Cilium / Flannel (CNI plugins)
- NetworkPolicy
- ip / iptables (node level)
- hubble (Cilium, optional)
- calicoctl (Calico, optional)

EXAMPLE NETWORKPOLICY (DEFAULT-DENY)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: backend
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

EXAMPLE ALLOW-LIST POLICY

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              name: frontend
      ports:
        - protocol: TCP
          port: 8080
```

TESTING COMMANDS

```
# Test connectivity from frontend to backend
kubectl exec -n frontend <pod> -- curl -v http://backend:8080

# Test DNS resolution
kubectl exec -n frontend <pod> -- nslookup
kubernetes.default.svc.cluster.local

# Check NetworkPolicies
kubectl get networkpolicies -A

# Describe policy
kubectl describe networkpolicy <name>

# View CNI (example: Calico)
calicoctl get networkpolicy -A

# Capture traffic on node (optional)
sudo tcpdump -i any host 10.244.2.20
```

TEST SCENARIOS

- 1 Apply default-deny policy and verify all traffic is blocked.
- 2 Allow specific ingress from frontend to backend.
- 3 Test egress rules to external services.
- 4 Verify DNS still works (allow CoreDNS egress).
- 5 Test namespace isolation (no cross-namespace traffic).
- 6 Check behavior with different CNIs (Calico, Cilium, Flannel).
- 7 Simulate a misconfigured policy and troubleshoot.
- 8 Validate real-world use cases (multi-tenant environments).

KEY TAKEAWAY

- ✓ NetworkPolicies provide layer 3/4 segmentation in Kubernetes.
- ✓ Use default-deny and explicit allow-lists for security.
- ✓ Always allow DNS (CoreDNS) when applying egress rules.
- ✓ Test both ingress and egress traffic.
- ✓ Understand CNI implementation limitations and features.
- ✓ Network segmentation reduces blast radius and improves cluster security.

NEXT STEP

- Explore advanced policies (namespace, pod, CIDR, FQDN).
- Test with eBPF-based CNI (Cilium).
- Integrate with service mesh (Istio) for L7 policies.
- Implement multi-tenant network segmentation.
- Monitor and audit network traffic.
- Build real-world scenarios (e.g. fintech, healthcare, SaaS).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta


VERIQTA
LEARN | BUILD | GROW

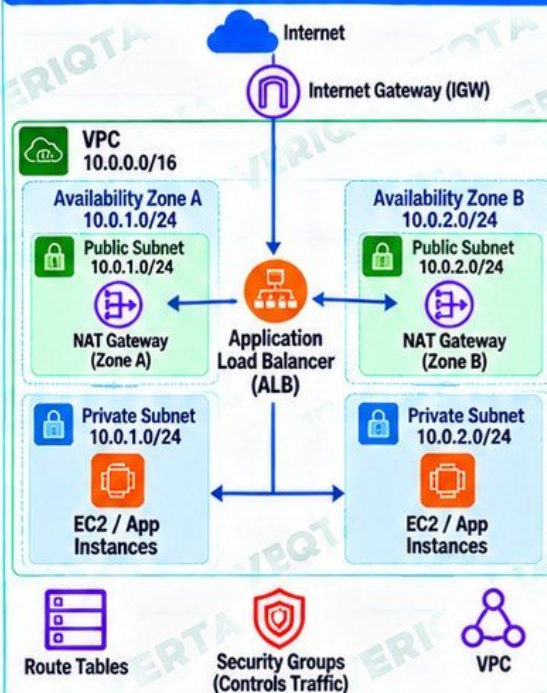
Build an AWS Multi-AZ VPC

Design public and private subnets across Availability Zones with route tables, Internet Gateway, NAT Gateway, security groups, and load balancing. Analyze availability, blast radius, and NAT cost trade-offs.

PROJECT GOAL

- Design a highly available, multi-AZ VPC in AWS.
- Create public and private subnets across multiple Availability Zones.
- Configure Internet Gateway (IGW) and NAT Gateway.
- Set up route tables and proper routing.
- Implement security groups for controlled access.
- Deploy an internet-facing load balancer and backend instances.
- Test connectivity from internet to application.
- Analyze availability, blast radius, and NAT cost trade-offs.
- Apply AWS best practices for production-ready networking.

ARCHITECTURE DIAGRAM



AWS SERVICES USED

- VPC
- Subnets (Public & Private)
- Internet Gateway (IGW)
- NAT Gateway
- Route Tables
- Security Groups
- Application Load Balancer (ALB)
- EC2 (or other compute)
- Elastic IP (for NAT Gateway)
- AWS CLI / Console
- VPC Flow Logs (optional)
- CloudWatch (optional)

KEY CONCEPTS

- Public vs Private Subnets
- Internet Gateway vs NAT Gateway
- Route Tables and Routing
- Security Groups (stateful)
- High Availability with Multi-AZ
- Blast Radius and Failure Scenarios
- NAT Gateway Cost Considerations
- Load Balancing and Health Checks
- VPC Design Best Practices

USEFUL AWS CLI COMMANDS

```
# Create VPC
aws ec2 create-vpc --cidr-block 10.0.0.0/16
# Create subnets
aws ec2 create-subnet --vpc-id vpc-xxxx --cidr-block 10.0.1.0/24 --availability-zone us-east-1a
# Create Internet Gateway and attach
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-xxxx --internet-gateway-id igw-xxxx
# Create NAT Gateway
aws ec2 create-nat-gateway --subnet-id subnet-xxxx --allocation-id eipalloc-xxxx
# Create route table and routes
aws ec2 create-route-table --vpc-id vpc-xxxx
# Create security group
aws ec2 create-security-group --group-name app-sg --description "App SG" --vpc-id vpc-xxxx
```

TESTING AND VALIDATION

```
# Test internet access from public instance
curl -I https://aws.amazon.com
# Test private instance internet access
curl -I https://aws.amazon.com
# Check route table
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-xxxx"
# Check security group rules
aws ec2 describe-security-groups --group-ids sg-xxxx
# Test load balancer DNS
curl -I http://<alb-dns-name>
# Simulate AZ failure (stop instances)
# and verify traffic still works
```

ANALYZE AND DISCUSS

- 1 What happens if one AZ goes down?
- 2 How does NAT Gateway affect cost?
- 3 Single vs Multi-AZ NAT Gateway trade-offs.
- 4 Blast radius of misconfigured security groups.
- 5 How route tables control traffic flow.
- 6 Load balancer health checks and failover.
- 7 How to reduce costs without sacrificing availability.
- 8 Real-world best practices and design recommendations.

COMMON ISSUES AND HOW TO FIX

Issue	Possible Cause	Fix
No internet access	Missing route to IGW/NAT	Update route table
Private instance cannot reach internet	Missing NAT Gateway or route	Create NAT Gateway and update route table
ALB not reachable	Security group or route issue	Check security groups and subnet configuration
High NAT costs	Excessive outbound traffic	Use VPC endpoints, NAT instance (where appropriate) or optimize traffic
Instances in one AZ unavailable	AZ failure or misconfiguration	Deploy across multiple AZs and use ALB



KEY TAKEAWAY

- ✓ Use multi-AZ design for high availability.
- ✓ Separate public and private subnets.
- ✓ Use NAT Gateway for private subnet internet access.
- ✓ Configure proper route tables and security groups.
- ✓ Understand cost implications of NAT Gateway.
- ✓ Test failure scenarios and analyze blast radius.
- ✓ Follow AWS Well-Architected best practices.



NEXT STEP

- Add VPC Flow Logs and analyze traffic.
- Implement VPC endpoints (S3, DynamoDB) to reduce NAT costs.
- Use Infrastructure as Code (Terraform).
- Deploy a real application (e.g. web app + database) in the VPC.
- Test AZ failure scenarios.
- Monitor with CloudWatch and set alarms.
- Explore advanced designs (Transit Gateway, multi-account, shared VPC).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

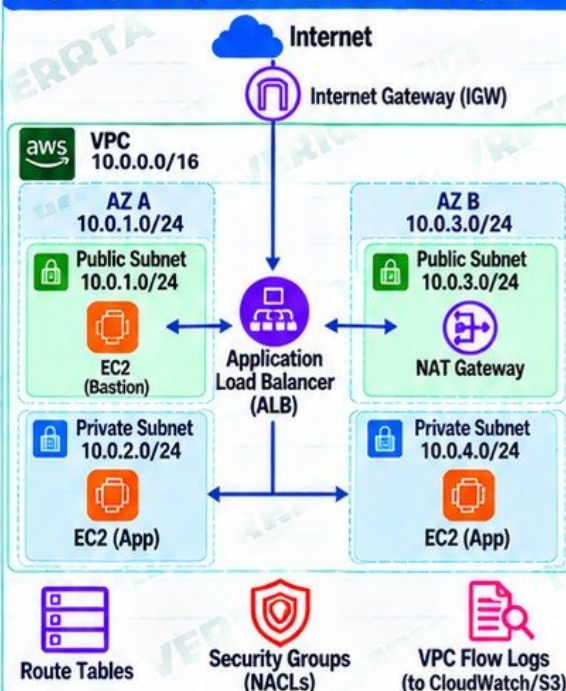
Troubleshoot AWS VPC Connectivity

Diagnose EC2 and application connectivity failures involving security groups, NACLs, routes, DNS, NAT, load balancers, and private subnets. Use VPC Flow Logs as evidence.

PROJECT GOAL

- Diagnose EC2 and application connectivity failures in a VPC.
- Troubleshoot security groups, NACLs, routes, DNS, NAT, load balancers, and private subnets.
- Use VPC Flow Logs to identify the root cause.
- Understand how traffic flows and where it gets blocked.
- Apply a structured troubleshooting methodology.
- Reproduce common VPC issues in a lab environment.
- Gain confidence in real-world incident investigation.

VPC TROUBLESHOOTING ARCHITECTURE



TOOLS USED

- AWS CLI / Console
- VPC Flow Logs
- Security Groups
- Network ACLs (NACLs)
- Route Tables
- Internet Gateway (IGW)
- NAT Gateway
- Application Load Balancer (ALB)
- EC2
- Route 53 (DNS)
- CloudWatch Logs
- VPC Reachability Analyzer
- traceroute / mtr
- curl / wget
- dig / nslookup
- tcpdump (on EC2)
- SSM Session Manager (optional)

USEFUL AWS CLI COMMANDS

```
# Check VPC and subnets
aws ec2 describe-vpcs
aws ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-xxxx"
# Check route tables
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-xxxx"
# Check security groups
aws ec2 describe-security-groups --group-ids sg-xxxx
# Check network ACLs
aws ec2 describe-network-acls --filters "Name=vpc-id,Values=vpc-xxxx"
# Check VPC Flow Logs
aws ec2 describe-flow-logs --filter "Name=resource-id,Values=vpc-xxxx"
# Check DNS resolution (from EC2)
dig example.com
nslookup example.com
# Test connectivity
curl -I http://<alb-dns-name>
traceroute <destination-ip>
```

COMMON CONNECTIVITY FAILURES

- 1 Security group blocking traffic.
- 2 NACL denying inbound or outbound traffic.
- 3 Missing or incorrect route table entry.
- 4 EC2 in private subnet without NAT.
- 5 DNS resolution failure (Route 53/Resolver).
- 7 ALB health check failures.
- 8 VPC endpoint or service access issues.
- 8 Asymmetric routing.
- 9 Using wrong port or protocol.
- 10 Traffic blocked (REJECT) in VPC Flow Logs.

TROUBLESHOOTING METHODOLOGY

- 1 Reproduce the issue and identify the source and destination.
- 2 Check security groups (inbound/outbound).
- 3 Check network ACLs (inbound/outbound).
- 4 Verify route tables and next hops.
- 5 Test DNS resolution.
- 6 Use VPC Flow Logs to confirm if traffic is allowed or rejected.
- 7 Validate NAT Gateway (if in private subnet).
- 8 Check load balancer and target health.
- 9 Use VPC Reachability Analyzer.
- 10 Isolate the root cause and implement a fix.

VPC FLOW LOG EXAMPLE (REJECT)

```
2 123456789012 eni-0abc123def4567890
10.0.2.10 8.8.8.8 443 6 10 1694659200 1694659260
REJECT -
```

- REJECT = traffic was blocked
- Source = 10.0.2.10 (private subnet)
- Destination = 8.8.8.8 (internet)
- Port = 443 (HTTPS)
- Use this to confirm if the issue is SG, NACL, or route.



KEY TAKEAWAY

- ✓ Follow the traffic path step by step (source → destination).
- ✓ Use VPC Flow Logs as evidence, not assumptions.
- ✓ Check SGs, NACLs, routes, DNS, NAT, and load balancers.
- ✓ Understand how private subnets access the internet.
- ✓ Use the right tools (Reachability Analyzer, Flow Logs, CloudWatch).
- ✓ A methodical approach reduces MTTR and prevents similar issues.



NEXT STEP

- Set up a VPC troubleshooting lab with intentional failures.
- Experiment with VPC Reachability Analyzer.
- Build monitoring and alerts for VPC Flow Logs.
- Explore VPC endpoints (S3, DynamoDB).
- Test hybrid connectivity (VPN / Direct Connect).
- Document common failure scenarios and runbooks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta


VERIQTA
LEARN | BUILD | GROW

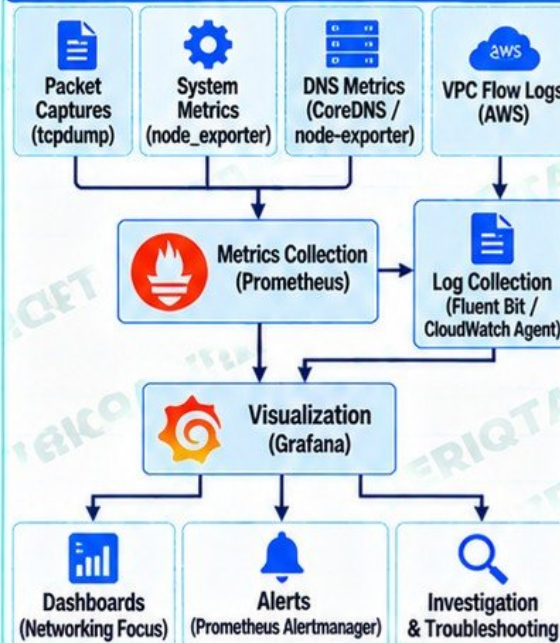
Build Network Observability

Combine packet captures, connection metrics, DNS metrics, flow logs, latency, retransmissions, connection counts, and application telemetry. Build a networking-focused troubleshooting dashboard.

PROJECT GOAL

- Collect and visualize network telemetry from multiple sources.
- Monitor packet-level data, connection metrics, DNS metrics, flow logs, latency, and retransmissions.
- Correlate network metrics with application performance.
- Build a centralized, networking-focused troubleshooting dashboard.
- Detect and diagnose common network issues (latency, packet loss, DNS failures, connection drops, etc.).
- Use open-source tools and real-world scenarios.
- Gain practical experience in network observability for cloud and Kubernetes environments.

OBSERVABILITY ARCHITECTURE



TOOLS USED

- tcpdump
- Wireshark (optional)
- Prometheus
- Grafana
- node_exporter
- cAdvisor (containers)
- CoreDNS metrics
- AWS VPC Flow Logs
- CloudWatch
- Fluent Bit / Fluentd
- Loki (optional)
- Alertmanager
- Blackbox Exporter
- kube-state-metrics
- Application metrics (e.g. Nginx, HTTP, DB)
- curl / wget
- ss / netstat
- iftop / nethogs (optional)

KEY METRICS TO COLLECT

- 1 Latency (RTT)
- 2 Packet loss
- 3 Retransmissions
- 4 Connection counts (active, new, closed)
- 5 DNS query rate, failures, latency
- 6 Network throughput (bytes in/out)
- 7 Flow logs (ACCEPT / REJECT)
- 8 HTTP response times (from application)
- 9 Error rates (4xx/5xx)
- 10 TLS handshake time (optional)

USEFUL COMMANDS

```

# Packet capture
sudo tcpdump -i any -nn host 10.0.0.10

# Check active connections
ss -tulnp

# Check retransmissions
netstat -s | grep retransmit

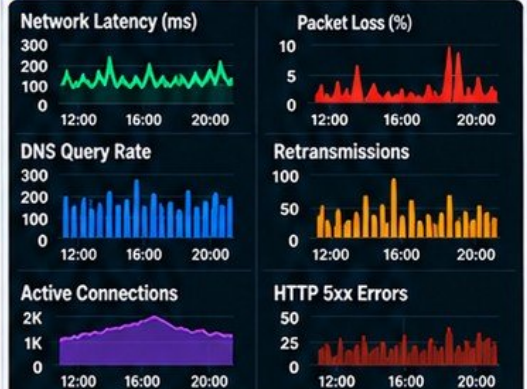
# DNS query test
dig example.com

# Check network metrics with iftop
sudo iftop -i eth0

# Query Prometheus metrics
curl http://localhost:9090/api/v1/query?query=up

# Check AWS VPC Flow Logs (example CLI)
aws ec2 describe-flow-logs --filter \
  "Name=resource-id,Values=fl-xxxxxxexx"
  
```

EXAMPLE GRAFANA DASHBOARD



TEST SCENARIOS

- 1 Simulate latency using tc.
- 2 Introduce packet loss.
- 3 Generate DNS failures.
- 4 Create high connection load.
- 5 Stop an application and observe network behavior.
- 6 Block traffic using security groups or NACLs.
- 7 Analyze VPC Flow Logs for rejected traffic.
- 8 Correlate network issues with application errors.



KEY TAKEAWAY

- ✓ Network observability combines multiple telemetry sources for faster troubleshooting.
- ✓ Correlate metrics, logs, and packet data to find the real root cause.
- ✓ Monitor DNS, latency, retransmissions, and connection counts.
- ✓ Use dashboards and alerts to detect issues early.
- ✓ Apply this approach in cloud, Kubernetes, and on-prem environments.
- ✓ A good network observability setup reduces MTTR and improves reliability.



NEXT STEP

- Add alerting rules for network anomalies.
- Integrate application metrics (e.g. Nginx, Envoy, database).
- Build dashboards for Kubernetes networking (Pod, Service, Ingress).
- Explore eBPF-based tools (BCC, Cilium).
- Implement distributed tracing to correlate network and application performance.
- Extend to multi-cloud environments (AWS, Azure, GCP).
- Document findings and create runbooks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

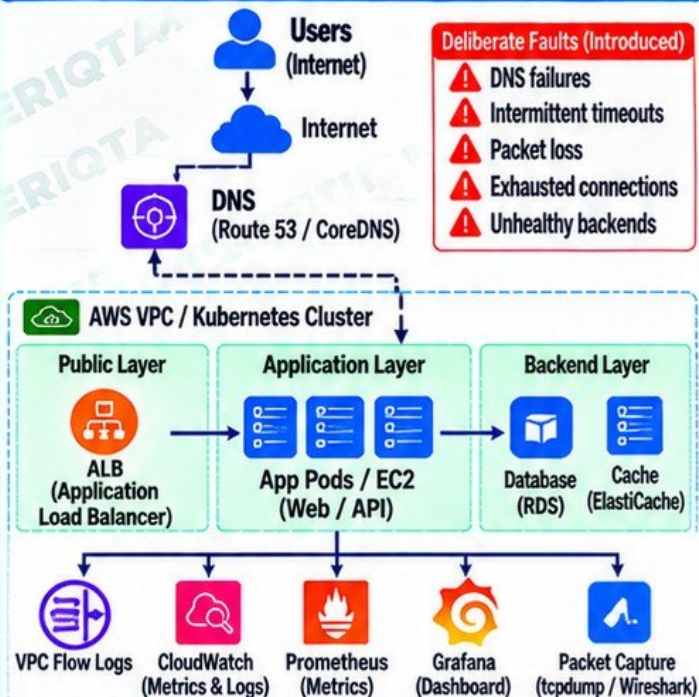
Final Project: Production Network Incident

Operate a multi-tier application with several deliberately introduced faults. Symptoms include intermittent timeouts, DNS failures, packet loss, exhausted connections, and unhealthy backends. Determine blast radius, gather evidence, identify root causes, mitigate, verify recovery, and design prevention measures.

PROJECT GOAL

- Investigate a realistic production network incident in a multi-tier application.
- Identify and analyze multiple network-related failures (DNS, timeouts, packet loss, connection exhaustion, unhealthy backends).
- Determine blast radius and customer impact.
- Gather and correlate evidence from multiple sources.
- Identify root causes and implement mitigation steps.
- Verify full recovery and validate system health.
- Design long-term prevention measures and monitoring improvements.
- Apply real-world incident response methodology.

MULTI-TIER APPLICATION ARCHITECTURE



TOOLS USED

- aws cli / console
- kubectl
- tcpdump
- wireshark
- dig / nslookup
- curl / wget
- netstat / ss
- iftop / nethogs
- traceroute / mtr
- prometheus
- grafana
- cloudwatch
- vpc flow logs
- k9s (optional)
- stern (optional)
- kubectl logs
- kubectl describe
- kubectl get events
- rds / database tools
- load balancer logs
- application logs
- jq (log parsing)

INCIDENT SYMPTOMS

- 1 Intermittent timeouts (user requests)
- 2 DNS resolution failures
- 3 Packet loss between services
- 4 Exhausted connections (too many open)
- 5 Unhealthy application backends
- 6 Increased latency (p95 / p99)
- 7 Partial or complete service disruption
- 8 Inconsistent behavior across regions/AZs

INVESTIGATION APPROACH

- 1 Confirm and understand the symptoms
- 2 Determine blast radius (what is affected)
- 3 Collect evidence (logs, metrics, packet captures)
- 4 Identify root causes (correlate data across layers)
- 5 Implement mitigation (short-term fix)
- 6 Verify recovery (end-to-end testing)
- 7 Design prevention measures (monitoring, alerts, architectural improvements)
- 8 Document timeline and lessons learned

USEFUL COMMANDS

```
# DNS checks
dig example.com
nslookup example.com

# Network connectivity
curl -I https://service.local
mtr <service-ip>

# Connection counts
ss -s
netstat -an | grep ESTABLISHED | wc -l

# Packet capture
sudo tcpdump -i eth0 -nn host <ip>

# Kubernetes debugging
kubectl get pods -A
kubectl describe pod <pod>
kubectl logs <pod>
```

EXAMPLE FAULTS (LAB SETUP)

- 1 Break CoreDNS (simulate DNS failures)
- 2 Add network latency and packet loss (tc/netem)
- 3 Limit connection pool (exhausted connections)
- 4 Make backend pods unhealthy
- 5 Misconfigure security group or NACL
- 6 Simulate intermittent route/DNS issues
- 7 Overload application to create timeouts
- 8 Introduce AZ failure (simulate node/pod failure)

KEY TAKEAWAY

- ✓ Real incidents have multiple concurrent issues.
- ✓ Use evidence, not assumptions.
- ✓ Correlate data across DNS, network, application and infrastructure.
- ✓ Determine blast radius before making changes.
- ✓ Communicate clearly during the incident.
- ✓ Verify full recovery, not just partial fixes.
- ✓ Implement monitoring, alerting and architectural improvements.
- ✓ Document lessons learned for future incidents.

NEXT STEP

- Build a complete production incident lab.
- Practice incident response with timed scenarios.
- Create dashboards for network, DNS and application health.
- Write an incident report template.
- Run regular game days.
- Expand to multi-region and multi-cloud scenarios.
- Share your project on GitHub.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA
LEARN | BUILD | GROW